

PerceptionLab

Dossier de Product Owning complet

Rust-powered ingestion and training platform for real-time computer vision models

Version 0.1 - Cadrage produit pour pass techno

Date: 15 juin 2026

Promesse produit: Upload data - build dataset - launch training - track metrics - export model - run inference.

Promesse portfolio: I can build production-grade infrastructure around machine learning models.



Note de cadrage

Ce document formalise le product owning complet de PerceptionLab. Il fixe le quoi et le pourquoi du projet avant la pass techno, qui devra ensuite fixer le comment: architecture Rust, schémas de données, queue, worker PyTorch, storage, export et stratégie de déploiement.

Le projet ne doit pas être présenté comme une simple démonstration de détection d'objets. Il doit être présenté comme une preuve de compétence ML systems: ingestion de données, dataset versioning, orchestration de jobs, entraînement PyTorch, registry de modèles, inférence et export mobile.

Angle stratégique: le repo doit prouver que le développeur sait construire l'infrastructure autour du modèle, pas seulement appeler un modèle existant.

Sommaire

Note de cadrage	2
1. Vision produit	5
2. Problème à résoudre	5
3. Positionnement	5
Ce que le projet n'est pas	5
Ce que le projet est	5
4. Objectifs produit	6
Objectifs secondaires	6
5. Utilisateurs cibles	6
6. Personas	6
Persona 1 - ML Builder	6
Persona 2 - Backend/Infra Engineer	7
Persona 3 - Recruteur technique	7
7. Proposition de valeur	7
8. Périmètre MVP	7
Inclus dans le MVP	7
Hors périmètre MVP	8
9. Périmètre V1, V2, V3	8
V1 - Core ML pipeline	8
V2 - Dataset quality + model registry avancé	8
V3 - Real-time perception demo	9
10. Parcours utilisateur principal	9
11. Parcours démo GitHub	9
12. Modules produit	10
Module A - Dataset Management	10
Module B - Sample Ingestion	10
Module C - Annotation Management	11
Module D - Dataset Versioning	11
Module E - Training Jobs	11
Module F - Metrics Tracking	12
Module G - Model Registry	12
Module H - Inference API	12
Module I - Model Export	13
Module J - Visual Overlay Demo	13
13. User stories par epic	14
14. Backlog priorisé	14
P0 - Obligatoire MVP	14
P1 - Fortement valorisant	15
P2 - Avancé	15
P3 - Bonus impressionnant	15
15. Exigences fonctionnelles	16
16. Exigences non fonctionnelles	16
Performance	16

Robustesse	16
Observabilité	16
Sécurité	17
Reproductibilité	17
17. Domain model	17
18. Statuts métier	18
19. Règles métier	18
20. API produit attendue	18
Endpoints MVP	18
Endpoints P1	18
21. Contrats API exemples	19
Création dataset	19
Création training job	19
Inference	19
22. Critères de réussite produit	20
Critères de réussite MVP	20
Critères de réussite portfolio	20
23. KPIs produit	20
KPIs techniques	20
KPIs portfolio	20
24. Définition du MVP livrable	21
25. Définition de Ready	21
26. Définition de Done	21
Pour les tâches ML	22
27. Risques produit	22
28. Hypothèses produit	22
29. Décisions produit recommandées	22
30. Roadmap proposée	23
31. README attendu	23
32. Démo finale cible	23
33. Ce que le projet doit prouver	24
34. Pitch entretien	24
Version courte	24
Version française	24
Version directe	25
35. Synthèse pour la pass techno	25

1. Vision produit

PerceptionLab doit démontrer qu'un modèle de computer vision ne vit pas dans un notebook isolé. Un vrai système ML a besoin d'une infrastructure autour: données, annotations, versioning, jobs, métriques, modèles, exports, monitoring et API.

Le produit poursuit deux objectifs. Le premier est de construire un outil réellement utile pour créer et entraîner des modèles de vision personnalisés. Le second est de produire un repo GitHub vitrine qui montre des compétences fortes en Rust, backend distribué, PyTorch, ML pipeline, API design, data engineering, MLOps et déploiement modèle.

```
Upload data -> build dataset -> launch training -> track metrics -> export model -> run inference.
```

```
I can build production-grade infrastructure around machine learning models.
```

2. Problème à résoudre

Beaucoup de projets IA restent superficiels: un notebook, un modèle pré-entraîné, une API wrapper ou une interface qui appelle un service externe. PerceptionLab traite un problème plus profond: construire une chaîne propre entre la donnée brute et un modèle exploitable en production.

- Les datasets sont souvent mal organisés: images, labels et métadonnées dispersés dans des dossiers non versionnés.
- Les jobs d'entraînement sont rarement traçables: on ne sait pas clairement avec quelles données, quels hyperparamètres et quelles métriques un modèle a été produit.
- Les modèles générés sont mal gérés: pas de registry, pas de versioning, pas d'export standard, pas de modèle promu.
- L'inférence est souvent séparée de l'entraînement: on entraîne dans un script puis on bricole un service différent pour tester.
- Les projets de vision n'exposent pas assez la compétence backend, alors que la valeur se situe dans le système complet.

3. Positionnement

PerceptionLab se positionne comme un projet de ML infrastructure appliquée à la computer vision. Le coeur du projet n'est pas la nouveauté du modèle, mais l'industrialisation de tout ce qui entoure le modèle.

Ce que le projet n'est pas

- Une simple app YOLO.
- Un notebook PyTorch isolé.
- Un clone complet de Roboflow.
- Une app mobile de détection.
- Un dashboard générique.

Ce que le projet est

- Un backend Rust d'ingestion et d'orchestration.
- Un worker PyTorch d'entraînement.
- Un système de dataset versioning.
- Un model registry.
- Une API d'inférence.

- Un pipeline d'export ONNX/CoreML.
- Une démo finale object detection + depth metadata.

PerceptionLab is a Rust + PyTorch ML infrastructure project for building, training, exporting and serving real-time computer vision models from versioned datasets.

4. Objectifs produit

L'objectif principal est de permettre à un utilisateur technique de créer un dataset, uploader des images, ajouter des annotations, lancer un entraînement, suivre les métriques, exporter un modèle et exécuter une inférence.

Objectifs secondaires

- Démontrer une architecture backend propre avec Rust.
- Démontrer un pipeline PyTorch réaliste.
- Démontrer une séparation claire entre API, stockage, queue, worker ML et registry.
- Démontrer une logique MLOps minimale mais crédible: dataset versioning, job tracking, métriques, artefacts, modèle exporté.
- Démontrer une capacité à produire une documentation haut niveau: README, diagrammes, API examples, benchmark et roadmap.

5. Utilisateurs cibles

Le premier utilisateur cible est un ML engineer indépendant qui veut entraîner rapidement un modèle de vision sur un dataset personnalisé sans tout refaire à la main.

Le deuxième utilisateur cible est un backend engineer qui veut intégrer une brique ML dans une infrastructure existante.

Le troisième utilisateur cible est un recruteur technique, staff engineer ou hiring manager qui regarde le GitHub et veut comprendre immédiatement que le projet n'est pas un simple wrapper IA.

Le quatrième utilisateur cible est le porteur du projet lui-même: le repo doit servir de preuve de compétence dans les candidatures, entretiens, missions freelance et discussions techniques.

6. Personas

Persona 1 - ML Builder

Le ML Builder veut créer un modèle de détection personnalisé. Il a des images, parfois des annotations, et veut lancer un fine-tuning PyTorch proprement.

- Créer un dataset
- Uploader des samples
- Ajouter ou importer des annotations
- Lancer un training job
- Consulter les métriques
- Récupérer un modèle entraîné
- Exporter en ONNX/CoreML
- Tester une image en inférence

Frustration actuelle: tout est fait dans des scripts isolés, sans traçabilité.

Persona 2 - Backend/Infra Engineer

Le Backend Engineer veut comprendre comment intégrer du ML dans un système robuste.

- API claire
- Schéma de données propre
- Stockage objet
- Queue async
- Logs
- Statuts de jobs
- Séparation API / workers
- Docker Compose
- Documentation d'architecture

Frustration actuelle: les projets ML sont souvent trop notebooks et pas assez systèmes.

Persona 3 - Recruteur technique

Le recruteur technique veut évaluer rapidement le niveau du projet.

- README clair
- Architecture diagram
- Démo reproductible
- Commandes curl
- Résultats visuels
- Benchmarks
- Choix techniques expliqués

Frustration actuelle: beaucoup de repos IA ne prouvent pas de compétence industrielle.

7. Proposition de valeur

Angle	Proposition de valeur
Utilisateur	PerceptionLab transforme des données visuelles brutes en modèles entraînés, versionnés, exportables et utilisables via API.
Technique	PerceptionLab démontre une architecture complète Rust + PyTorch pour industrialiser un pipeline computer vision.
Portfolio	PerceptionLab prouve la capacité à construire le système autour du modèle, pas seulement à appeler un modèle existant.

8. Périmètre MVP

Le MVP doit être ambitieux mais maîtrisé. Il doit montrer toute la chaîne sans essayer de devenir une plateforme complète dès la première version.

Inclus dans le MVP

- Création de datasets
- Upload d'images
- Stockage fichier dans MinIO ou filesystem local compatible S3
- Stockage metadata dans PostgreSQL

- Création d'annotations object detection
- Création de training jobs
- Queue de jobs
- Worker PyTorch
- Fine-tuning d'un modèle de détection
- Sauvegarde checkpoints et métriques
- Model registry simple
- Endpoint d'inférence
- Export ONNX
- Documentation complète
- Docker Compose

Hors périmètre MVP

- Interface d'annotation avancée type CVAT
- Multi-user complexe
- Paiement
- Auth OAuth complète
- Training distribué multi-GPU
- AutoML
- Support complet vidéo temps réel
- Monitoring production Prometheus/Grafana obligatoire
- Versioning complexe type DVC complet

9. Périmètre V1, V2, V3

V1 - Core ML pipeline

- Créer dataset
- Uploader images
- Ajouter annotations
- Lancer training job
- Voir statut job
- Voir métriques
- Enregistrer modèle
- Faire inférence
- Exporter ONNX

La V1 doit être démontrable localement avec docker compose up, puis quelques commandes curl ou un petit client CLI.

V2 - Dataset quality + model registry avancé

- Dataset versions
- Train/val/test split configurable
- Import YOLO format

- Export YOLO format
- Comparaison de modèles
- Promotion d'un modèle en production
- CoreML export
- Visualisation overlays
- Métriques par classe

V3 - Real-time perception demo

- Client mobile ou web caméra
- Streaming frame-by-frame
- Inférence temps réel
- Overlay bbox
- Depth metadata optionnelle
- Tracking simple
- Latency benchmark

10. Parcours utilisateur principal

Le parcours principal doit être simple, lisible et démontrable de bout en bout.

1. POST /datasets
L'utilisateur crée un dataset et reçoit un dataset_id.
2. POST /datasets/{dataset_id}/samples
Chaque image uploadée devient un sample.
3. POST /samples/{sample_id}/annotations
L'utilisateur ajoute une classe et une bounding box.
4. POST /datasets/{dataset_id}/versions
La version fige les samples et annotations utilisés.
5. POST /training-jobs
Le job référence une dataset_version et une configuration modèle.
6. GET /training-jobs/{job_id}
Statuts: queued, running, succeeded, failed, cancelled.
7. GET /training-jobs/{job_id}/metrics
L'utilisateur consulte les métriques.
8. GET /models/{model_id}
L'utilisateur récupère le modèle généré.
9. POST /models/{model_id}/infer
L'utilisateur lance une inférence.
10. POST /models/{model_id}/exports
L'utilisateur exporte en onnx, coreml ou torchscript.

11. Parcours démo GitHub

Ce parcours est stratégique parce que le projet est aussi un asset de recrutement. La démo doit pouvoir se raconter en moins de deux minutes.

1. Je démarre la stack avec Docker Compose.
2. Je crée un dataset desk-objects-v1.
3. J'upload quelques images.
4. J'ajoute ou j'importe des annotations YOLO.
5. Je lance un job PyTorch.
6. Je vois les métriques évoluer.
7. Je récupère le modèle entraîné.
8. Je lance une inférence sur une image de test.
9. L'API retourne les boxes + confidence.
10. Je génère un overlay visuel.

Message à faire passer: This is not a model demo. This is ML infrastructure.

12. Modules produit

Module A - Dataset Management

Ce module permet de créer, lister, mettre à jour et consulter des datasets.

Objet	Champs
Dataset	id, name, description, task_type, classes, created_at, updated_at, status
Tâche MVP	object_detection
Tâches futures	image_classification, segmentation, depth_estimation, multimodal_detection

- Créer un dataset
- Lister les datasets
- Voir le détail
- Ajouter une liste de classes
- Consulter samples et annotations
- Créer une version figée

Critères d'acceptation: création avec nom, description et classes; identifiant stable; liste et détail disponibles; stats accessibles; impossibilité de lancer un training job sans classes définies.

Module B - Sample Ingestion

Ce module permet d'uploader des images et de stocker leurs métadonnées.

Objet	Champs
Sample	id, dataset_id, storage_uri, filename, mime_type, width, height, size_bytes, checksum, source, metadata, created_at
Formats MVP	jpg, jpeg, png, webp
Formats futurs	mp4, mov, heic, depth map

- Upload image
- Validation format
- Calcul checksum
- Détection doublons basique
- Extraction width/height
- Stockage objet
- Création metadata PostgreSQL

Critères d'acceptation: upload dans un dataset existant; stockage effectif; métadonnées persistées; rejet des fichiers non-image ou trop lourds; retour d'un sample_id.

Module C - Annotation Management

Ce module permet d'ajouter des bounding boxes aux samples.

Objet	Champs
Annotation	id, sample_id, dataset_id, class_name, class_id, bbox, format, confidence, source, created_at
Format bbox interne	x, y, width, height normalisés entre 0 et 1
Sources	manual, imported, model_generated, corrected

- Ajouter annotation
- Lister annotations d'un sample
- Supprimer annotation
- Importer YOLO
- Exporter YOLO
- Valider bbox

Critères d'acceptation: annotation liée à un sample; classe existante dans le dataset; bbox dans les limites; conversion interne vers YOLO; import YOLO simple.

Module D - Dataset Versioning

Ce module fige un état du dataset pour garantir la reproductibilité.

Objet	Champs
DatasetVersion	id, dataset_id, version_name, sample_count, annotation_count, classes_snapshot, split_config, created_at, created_by
Exemple	desk-objects-v1@2026-06-15

- Créer version
- Lister versions
- Voir détail version
- Associer version à training job
- Empêcher modification d'une version figée

Critères d'acceptation: un training job référence une dataset_version; la version conserve classes, samples et annotations; les changements ultérieurs sur le dataset ne modifient pas la version existante.

Module E - Training Jobs

Ce module permet de créer et suivre des entraînements PyTorch.

Objet	Champs
Trainingjob	id, dataset_version_id, model_family, base_model, status, hyperparameters, created_at, started_at, finished_at, error_message, output_model_id
Statuts	queued, running, succeeded, failed, cancelled
Hyperparamètres MVP	epochs, batch_size, image_size, learning_rate, optimizer, augmentation

- Créer training job
- Mettre job en queue

- Worker récupère job
- Worker exécute training
- Worker met à jour statut
- Worker écrit métriques
- Worker crée model artifact

Critères d'acceptation: création depuis dataset version; statut queued; passage running; issue succeeded ou failed; métriques sauvegardées; modèle créé en fin de succès.

Module F - Metrics Tracking

Ce module permet de stocker et lire les métriques d'entraînement.

- train_loss
- val_loss
- precision
- recall
- mAP50
- mAP50_95
- epoch
- learning_rate
- duration_seconds

Critères d'acceptation: un job réussi possède une métrique finale; les métriques sont associées au job_id; elles sont récupérables via API; le résumé affiche la meilleure epoch.

Module G - Model Registry

Le Model Registry stocke les modèles produits par les jobs d'entraînement.

Objet	Champs
Model	id, name, version, training_job_id, dataset_version_id, model_family, artifact_uri, metrics_summary, status, created_at
Statuts	candidate, validated, promoted, archived

- Créer modèle après training
- Lister modèles
- Voir détail modèle
- Télécharger artefact
- Promouvoir modèle
- Archiver modèle

Critères d'acceptation: création uniquement depuis un job réussi; référence au dataset versionné; artifact_uri; métriques principales exposées; règle V2 pour un seul modèle promoted par famille/dataset.

Module H - Inference API

Ce module permet de tester un modèle sur une image.

Input: image file, model_id, options

Output:

```
{
  "model_id": "model_123",
  "latency_ms": 42,
  "detections": [
    {
      "class_id": 0,
      "class_name": "cup",
      "confidence": 0.89,
      "bbox": { "x": 0.36, "y": 0.48, "width": 0.28, "height": 0.31 }
    }
  ]
}
```

- Upload image pour inférence
- Chargement modèle
- Préprocessing image
- Inférence
- Post-processing NMS
- Retour JSON
- Génération overlay optionnelle

Critères d'acceptation: image envoyée à un modèle existant; liste de détections; classe, confiance et bbox; latence; rejet model_id inexistant ou non disponible.

Module I - Model Export

Ce module convertit un modèle vers un format déployable.

- Format MVP: onnx
- Formats V2: coreml, torchscript

Un export contient: id, model_id, format, artifact_uri, status, created_at, error_message.

Critères d'acceptation: demande d'export ONNX; artefact enregistré; statut disponible; erreur lisible en cas d'échec.

Module J - Visual Overlay Demo

Ce module rend le résultat parlant. Il prend une image et des détections, puis génère une image annotée avec bounding boxes, nom de classe, confiance et distance optionnelle.

```
cup 89% - 0.4 m
book 74% - 1.2 m
```

Critères d'acceptation: overlay générable depuis une inférence; boxes visibles; labels lisibles; class_name et confiance; affichage de distance_m si disponible dans metadata.

13. User stories par epic

Epic	User story	Critères d'acceptation	Priorité
Dataset creation	En tant qu'utilisateur, je veux créer un dataset pour organiser mes données d'entraînement.	Créer avec name/task_type/classes; recevoir dataset_id; récupérer et lister.	P0
Image ingestion	Je veux uploader des images dans un dataset pour construire mon corpus.	Image valide stockée; métadonnées sauvegardées; samples listables; formats invalides rejetés.	P0
Annotation	Je veux annoter mes images avec des bounding boxes.	Ajouter/lister/supprimer bbox; classe existante; bbox valide.	P0
Dataset versioning	Je veux figer une version pour garantir la reproductibilité.	Version capture samples, annotations et classes; training job référence une version immuable.	P0
Training orchestration	Je veux lancer un entraînement PyTorch depuis une version.	Job queued, running, succeeded/failed; statut consultable.	P0
Metrics	Je veux suivre les métriques du training.	Métriques par epoch; récupération API; meilleur score final.	P0
Model registry	Je veux retrouver les modèles entraînés et leurs artefacts.	Job réussi crée modèle; références conservées; artefact listable et téléchargeable.	P0
Inference	Je veux tester un modèle sur une image.	Retour detections avec bbox, classe, confidence et latence.	P0
Export ONNX	Je veux exporter mon modèle pour le déployer ailleurs.	Export ONNX créé, lié au modèle et téléchargeable.	P1
Demo overlay	Je veux générer une image annotée pour visualiser les résultats.	Boxes dessinées, labels lisibles, résultat sauvegardé ou retourné.	P1

14. Backlog priorisé

P0 - Obligatoire MVP

- Créer API Rust avec healthcheck
- Créer schéma PostgreSQL initial
- Créer endpoint POST /datasets et GET /datasets
- Créer endpoint POST /datasets/{id}/samples
- Connecter storage fichier
- Créer endpoints annotations
- Créer endpoint GET /datasets/{id}/stats
- Créer endpoint POST /datasets/{id}/versions
- Créer endpoint POST /training-jobs
- Créer table training_jobs
- Créer queue de jobs
- Créer worker PyTorch minimal
- Créer training loop ou wrapper modèle
- Sauvegarder métriques

- Créer model registry minimal
- Créer endpoint GET /models
- Créer endpoint POST /models/{id}/infer
- Créer docker-compose
- Créer README de démarrage

P1 - Fortement valorisant

- Import annotations YOLO
- Export annotations YOLO
- Export modèle ONNX
- Génération overlay image
- Métriques par classe
- Endpoint GET /training-jobs/{id}/metrics
- OpenAPI/Swagger
- CLI simple
- Seed dataset de démonstration
- Benchmark latence inference

P2 - Avancé

- CoreML export
- Dataset split configurable
- Comparaison modèles
- Promotion modèle
- Auth API key
- Dashboard web minimal
- Streaming logs training
- Support vidéo
- Support depth metadata
- Tracking simple

P3 - Bonus impressionnant

- Real-time camera client
- Mobile iOS prototype
- Segmentation model
- Depth-aware detections
- Model drift report
- Human-in-the-loop correction
- Auto-labeling avec modèle existant

15. Exigences fonctionnelles

ID	Exigence
FR-001	Créer un dataset avec un nom unique, un type de tâche et une liste optionnelle de classes.
FR-002	Uploader une image dans un dataset existant.
FR-003	Valider le type MIME, la taille maximale et les dimensions de l'image.
FR-004	Stocker le fichier dans un storage objet ou filesystem local abstrait derrière une interface.
FR-005	Extraire et sauvegarder filename, width, height, size et checksum.
FR-006	Ajouter une annotation bounding box à un sample.
FR-007	Vérifier que les coordonnées bbox sont valides et que la classe existe.
FR-008	Créer une version immuable d'un dataset.
FR-009	Créer un job d'entraînement à partir d'une dataset_version.
FR-010	Placer le job dans une queue consommée par un worker.
FR-011	Le worker récupère les jobs, charge les données, lance l'entraînement et met à jour le statut.
FR-012	Sauvegarder les métriques d'entraînement.
FR-013	Sauvegarder l'artefact modèle à la fin d'un job réussi.
FR-014	Référencer chaque modèle dans un registry consultable via API.
FR-015	Exécuter une inférence sur une image avec un modèle sélectionné.
FR-016	Exporter au moins un modèle au format ONNX.
FR-017	Générer une visualisation des détections sur image.

16. Exigences non fonctionnelles

Performance

L'API d'ingestion doit accepter plusieurs uploads simultanés sans bloquer le serveur. Le training doit être asynchrone: aucun entraînement ne doit bloquer une requête HTTP. L'inférence MVP peut être synchrone, mais doit retourner la latence.

- Upload image < 2s pour une image standard locale
- Création job < 500ms hors queue
- Inférence locale < 500ms sur CPU pour petit modèle
- Inférence GPU selon disponibilité

Robustesse

- Le système survit à un worker indisponible: les jobs restent queued.
- Un job échoué stocke une erreur lisible.
- Les fichiers uploadés ne créent pas de metadata orpheline si le stockage échoue.
- Les statuts de job restent cohérents.

Observabilité

- Logs structurés
- request_id pour les requêtes importantes
- Historique minimal d'événements par job

- API logs
- Worker logs
- Job status
- Error message
- Metrics table

Sécurité

- MVP local sans auth obligatoire
- Validation stricte des fichiers
- Limite taille upload
- Pas d'exécution arbitraire depuis payload utilisateur
- Sanitization filename
- V2: API key, rate limiting, ownership, permissions dataset/model

Reproductibilité

- dataset_version_id
- model_family
- base_model
- hyperparameters
- code_version optionnelle
- created_at

La reproductibilité parfaite n'est pas obligatoire en MVP, mais la traçabilité doit être visible.

17. Domain model

Objets métier principaux:

Dataset
Sample
Annotation
DatasetVersion
TrainingJob
TrainingMetric
Model
ModelExport
InferenceRun
Artifact

Relations:

Dataset 1 -> N Sample
Sample 1 -> N Annotation
Dataset 1 -> N DatasetVersion
DatasetVersion 1 -> N TrainingJob
TrainingJob 1 -> 0..1 Model
TrainingJob 1 -> N TrainingMetric
Model 1 -> N ModelExport
Model 1 -> N InferenceRun

18. Statuts métier

Objet	Statuts
Dataset	draft, ready, archived
TrainingJob	queued, running, succeeded, failed, cancelled
Model	candidate, validated, promoted, archived
Export	queued, running, succeeded, failed

19. Règles métier

- Un dataset peut être modifié tant qu'il n'est pas archivé.
- Une dataset_version est immuable.
- Un training job doit toujours utiliser une dataset_version, pas un dataset directement.
- Un sample doit appartenir à un dataset.
- Une annotation doit appartenir à un sample.
- Une annotation doit référencer une classe existante.
- Un modèle ne peut être créé que depuis un training job réussi.
- Un export ne peut être créé que depuis un modèle existant.
- Un modèle archivé ne doit pas être utilisé par défaut pour l'inférence.
- Un job failed doit conserver son message d'erreur.

20. API produit attendue

Endpoints MVP

```

GET    /health
POST   /datasets
GET    /datasets
GET    /datasets/{dataset_id}
GET    /datasets/{dataset_id}/stats
POST   /datasets/{dataset_id}/samples
GET    /datasets/{dataset_id}/samples
POST   /samples/{sample_id}/annotations
GET    /samples/{sample_id}/annotations
POST   /datasets/{dataset_id}/versions
GET    /datasets/{dataset_id}/versions
POST   /training-jobs
GET    /training-jobs
GET    /training-jobs/{job_id}
GET    /training-jobs/{job_id}/metrics
GET    /models
GET    /models/{model_id}
POST   /models/{model_id}/infer
POST   /models/{model_id}/exports
GET    /models/{model_id}/exports

```

Endpoints P1

```

POST   /datasets/{dataset_id}/import/yolo
GET    /datasets/{dataset_id}/export/yolo
POST   /inference-runs/{run_id}/overlay
GET    /artifacts/{artifact_id}/download

```

21. Contrats API exemples

Création dataset

```
Request:
{
  "name": "desk-objects-v1",
  "description": "Dataset for detecting desk objects from iPhone captures",
  "task_type": "object_detection",
  "classes": ["cup", "book", "phone", "keyboard", "mouse"]
}

Response:
{
  "id": "ds_01hxyz",
  "name": "desk-objects-v1",
  "task_type": "object_detection",
  "classes": ["cup", "book", "phone", "keyboard", "mouse"],
  "status": "draft",
  "created_at": "2026-06-15T12:00:00Z"
}
```

Création training job

```
Request:
{
  "dataset_version_id": "dsv_01hxyz",
  "model_family": "yolo",
  "base_model": "yololln",
  "hyperparameters": {
    "epochs": 50,
    "batch_size": 16,
    "image_size": 640,
    "learning_rate": 0.001
  }
}

Response:
{
  "id": "job_01hxyz",
  "status": "queued",
  "dataset_version_id": "dsv_01hxyz",
  "model_family": "yolo",
  "created_at": "2026-06-15T12:05:00Z"
}
```

Inference

```
Request:
multipart/form-data
image=@test.jpg
confidence_threshold=0.25

Response:
{
  "model_id": "mdl_01hxyz",
  "latency_ms": 48,
  "detections": [
    {
      "class_id": 0,
      "class_name": "cup",
      "confidence": 0.89,
      "bbox": { "x": 0.36, "y": 0.48, "width": 0.28, "height": 0.31 },
      "distance_m": 0.4
    }
  ]
}
```

22. Critères de réussite produit

Le projet est réussi si une personne peut cloner le repo, lancer la stack et comprendre la valeur en moins de dix minutes.

Critères de réussite MVP

- docker compose up fonctionne
- L'API Rust démarre
- PostgreSQL démarre
- Le storage démarre
- Le worker démarre
- Un dataset peut être créé
- Des images peuvent être uploadées
- Des annotations peuvent être ajoutées ou importées
- Un training job peut être lancé
- Un modèle est produit
- Une inférence retourne des detections
- Un overlay peut être généré
- Le README explique clairement l'architecture

Critères de réussite portfolio

- Le repo montre Rust de manière sérieuse
- Le repo montre PyTorch de manière sérieuse
- Le repo montre une architecture de pipeline
- Le repo montre des choix techniques justifiés
- Le repo montre une démo visuelle
- Le repo donne envie de poser des questions en entretien

23. KPIs produit

KPIs techniques

- Nombre d'images ingérées
- Temps moyen upload image
- Nombre de jobs lancés
- Taux de jobs réussis
- Durée moyenne training
- mAP50 final
- Latence inference
- Taille modèle exporté
- Temps export ONNX

KPIs portfolio

- Temps nécessaire pour comprendre le projet
- Temps nécessaire pour lancer la démo

- Clarté du README
- Présence diagramme architecture
- Présence GIF/demo image
- Présence commandes curl
- Présence benchmark
- Présence roadmap

24. Définition du MVP livrable

- Un repo GitHub propre
- Un backend Rust
- Un worker Python/PyTorch
- Une base PostgreSQL
- Un storage objet ou local
- Une queue
- Un Docker Compose
- Une documentation OpenAPI
- Un README orienté recruteur technique
- Un dataset exemple minimal
- Un modèle entraînable
- Une inférence démontrable
- Une image overlay générée

Le MVP n'a pas besoin d'être parfait. Il doit être crédible, exécutable et bien documenté.

25. Définition de Ready

Une tâche est prête à être développée si elle possède un objectif clair, un endpoint ou composant ciblé, un input attendu, un output attendu, des critères d'acceptation, une priorité et une dépendance identifiée.

Exemple: Créer POST /datasets

Objectif:

Permettre la création d'un dataset object_detection.

Input:

name, description, task_type, classes

Output:

dataset_id, status, created_at

Acceptance criteria:

- Retourne 201 si payload valide
- Refuse name vide
- Refuse task_type inconnu
- Persiste en PostgreSQL
- Test d'intégration présent

26. Définition de Done

Une tâche est done si le code est implémenté, le comportement est testé, les erreurs principales sont gérées, l'endpoint est documenté, les logs utiles existent, les critères d'acceptation sont respectés, le code est lisible et la documentation est mise à jour si nécessaire.

Pour les tâches ML

- Le script fonctionne sur dataset exemple
- Les artefacts sont sauvegardés
- Les métriques sont produites
- Les erreurs sont explicites
- Les dépendances sont documentées

27. Risques produit

Risque	Mitigation
Le projet devient trop gros.	Garder le MVP API-first, sans dashboard complexe au début.
Le training prend trop de temps ou demande un GPU.	Prévoir un mode mock training ou tiny dataset demo pour la démo locale.
Le modèle n'est pas très performant.	Assumer que l'objectif MVP est la plateforme, pas le score SOTA; utiliser un dataset simple et un modèle pré-entraîné.
La partie Rust et la partie Python deviennent mal couplées.	Définir un contrat job clair via queue + DB + artifacts.
Le README ne vend pas assez bien le projet.	L'écrire comme une page produit technique avec diagramme, screenshots, commandes et scénario.
L'inférence Rust avec modèle ML devient trop complexe.	En MVP, servir l'inférence par Python ou un service séparé; Rust orchestre. Optimiser plus tard avec ONNX Runtime côté Rust.

28. Hypothèses produit

- Les utilisateurs techniques préfèrent une plateforme reproductible à une simple démo UI.
- Le recruteur valorisera davantage une architecture Rust + PyTorch bien documentée qu'un front très joli.
- Un pipeline local Docker Compose est suffisant pour prouver la compétence.
- Un modèle pré-entraîné fine-tuné est suffisant pour démontrer la maîtrise PyTorch.
- La valeur principale est dans la chaîne de bout en bout.

29. Décisions produit recommandées

Décision	Justification
Commencer sans dashboard	Un dashboard coûte du temps et détourne du vrai signal technique. API + README + OpenAPI + demo images suffisent pour V1.
Rust pour l'API, Python pour le training	Rust montre le sérieux backend; Python/PyTorch montre la compétence ML.
Dataset versioning simple mais obligatoire	C'est une preuve MLOps forte. Même simple, cela fait beaucoup plus sérieux qu'un dossier data/.
Inference simple en P0, ONNX en P1	Il faut d'abord fermer la boucle produit. L'optimisation vient ensuite.
Seed dataset minimal	Le recruteur doit pouvoir lancer la démo sans créer lui-même un dataset complet.

30. Roadmap proposée

Semaine	Objectif	Livrables
1	API foundation	Axum server, PostgreSQL connection, migrations, healthcheck, Dataset CRUD minimal, sample upload, storage abstraction, Docker Compose initial
2	Annotation + versioning	Annotation endpoints, validation bbox, dataset stats, dataset versions, import YOLO simple, seed dataset, tests API principaux
3	Training pipeline	Training jobs, queue, worker Python, dataset materialization, training script, metrics writing, job status lifecycle
4	Model registry + inference	Model registry, artifact storage, inference endpoint, detection JSON, overlay generation, ONNX export initial
5	Polish portfolio	README premium, architecture diagram, demo GIF, curl examples, benchmark, model card, roadmap, technical decisions document

31. README attendu

Le README doit être pensé comme une page produit technique, pas comme une note personnelle.
Structure recommandée:

```
# PerceptionLab

## What is it?
## Why this project?
## Architecture
## Features
## Quickstart
## API examples
## Training pipeline
## Model registry
## Inference demo
## Benchmarks
## Tech stack
## Roadmap
## Technical decisions
```

Most computer vision demos stop at inference. PerceptionLab focuses on the infrastructure required before and after the model: ingestion, dataset versioning, training orchestration, metrics, registry and export.

32. Démo finale cible

Élément	Cible
Dataset	desk-objects-v1
Classes	cup, book, phone, keyboard, mouse
Input	photo de bureau
Output	image annotée + JSON

```
Output visuel:  
cup 89%  
book 76%  
phone 91%  
  
Avec depth metadata optionnelle:  
cup 89% - 0.4 m  
book 76% - 1.1 m  
  
Fichiers à prévoir dans le repo:  
docs/demo/input.jpg  
docs/demo/output_overlay.jpg  
docs/demo/inference_response.json
```

33. Ce que le projet doit prouver

- Rust API design
- Async backend
- PostgreSQL schema design
- File ingestion
- Object storage
- Job orchestration
- Queue architecture
- Python worker integration
- PyTorch training
- Computer vision pipeline
- Model artifacts
- Model registry
- Inference serving
- Export ONNX/CoreML
- Dockerized local infrastructure
- Technical documentation

C'est exactement le trou à combler dans un GitHub orienté IA/fullstack: montrer que le développeur sait construire l'infrastructure autour de l'intelligence artificielle.

34. Pitch entretien

Version courte

PerceptionLab is a Rust and PyTorch platform that turns raw visual data into trainable, versioned and deployable computer vision models. It includes dataset ingestion, annotation storage, dataset versioning, async training jobs, metrics tracking, model registry, inference API and ONNX export.

Version française

PerceptionLab est une plateforme Rust + PyTorch qui prend des données visuelles brutes et les transforme en modèles de computer vision entraînés, versionnés et déployables. Le projet couvre l'ingestion, les annotations, le versioning dataset, les jobs d'entraînement asynchrones, les métriques, le registry de modèles, l'inférence et l'export ONNX/CoreML.

Version directe

Ce projet montre que je ne fais pas seulement tourner un modèle. Je construis toute l'infrastructure nécessaire pour l'entraîner, le suivre, le versionner et le servir.

35. Synthèse pour la pass techno

La pass techno doit répondre aux questions d'architecture et d'implémentation que le product owning ne tranche pas volontairement.

- Quel framework Rust ?
- Quelle stratégie storage ?
- Quelle DB schema ?
- Quelle queue ?
- Comment Rust communique avec Python ?
- Où vivent les artefacts ?
- Comment versionner un dataset ?
- Comment matérialiser un dataset pour PyTorch ?
- Quel modèle de détection utiliser ?
- Comment sauvegarder les métriques ?
- Comment gérer les statuts de job ?
- Comment servir l'inférence ?
- ONNX côté Python ou côté Rust ?
- Comment packager en Docker Compose ?
- Quels tests pour prouver que ça marche ?

Le Product Owning fixe le quoi et le pourquoi. La pass techno devra fixer le comment. La direction produit est claire: PerceptionLab doit être une preuve de compétence ML infrastructure, pas une simple démo computer vision.